

Relatório: Paralelização do Algoritmo KNN com OpenMP

Instituição: Universidade do Estado de Santa Catarina (UDESC)

Disciplina: Sistemas Paralelos

Aluno: Eduardo Lopes Jonker

Objetivo: Implementar e analisar a versão paralela do algoritmo K-Nearest Neighbors (KNN) utilizando a biblioteca OpenMP para ganho de desempenho em processamento de dados massivos.

1. Implementação do Código

Abaixo seguem os códigos-fonte utilizados no projeto, estruturados para suportar execução paralela.

1.1. Cabeçalho e Estruturas (`algorithm/knn.h`)

```
```c
```

```
include <stdio.h>
```

```
include <stdlib.h>
```

```
typedef struct {
```

```
 float x;
```

```
 float y;
```

```
} Point;
```

```
typedef struct {
```

```
 char label;
```

```
 int length;
```

```
 Point points;
```

```
} Group;
```

```
// Funções de parsing e auxiliares omitidas para brevidade,
```

```
// mas presentes na implementação original.
```

```
float euclidean_distance_no_sqrt (Point a, Point b) {
 return ((b.x - a.x) (b.x - a.x)) + ((b.y - a.y) (b.y - a.y));
}
...
```

## 1.2. Lógica Principal e Paralelismo (`algorithm/knn.c`)

```
```c
```

```
include "knn.h"
```

```
include <omp.h>
```

```
char knn(int n_groups, Group groups, int k, Point to_evaluate) {  
    char labels = (char ) malloc(sizeof(char) k);  
    float distances = (float ) malloc(sizeof(float) k);
```

```
    // Inicialização do ranking global
```

```
    for (int i = 0; i < k; i++) {  
        labels[i] = -1;  
        distances[i] = -1.0f;  
    }
```

```
    pragma omp parallel
```

```
    {
```

```
        // Espaço local de cada thread para evitar contenção
```

```
        char local_labels = (char ) malloc(sizeof(char) k);  
        float local_distances = (float ) malloc(sizeof(float) k);  
        for (int i = 0; i < k; i++) local_distances[i] = -1.0f;
```

```
        pragma omp for schedule(dynamic)
```

```
        for (int i = 0; i < n_groups; i++) {  
            Group g = groups[i];
```

```

for (int j = 0; j < g.length; j++) {
    float d = euclidean_distance_no_sqrt(to_evaluate, g.points[j]);

    // Inserção ordenada no Top-K local
    if (local_distances[k-1] != -1.0f && d >= local_distances[k-1]) continue;
    int x = k - 1;
    while (x > 0 && (local_distances[x-1] == -1.0f || d < local_distances[x-1])) {
        local_distances[x] = local_distances[x-1];
        local_labels[x] = local_labels[x-1];
        x--;
    }
    local_distances[x] = d;
    local_labels[x] = g.label;
}
}

// Mesclagem segura no ranking global
#pragma omp critical
{
    for (int i = 0; i < k; i++) {
        if (local_distances[i] == -1.0f) break;
        float d = local_distances[i];
        if (distances[k-1] != -1.0f && d >= distances[k-1]) continue;
        int x = k - 1;
        while (x > 0 && (distances[x-1] == -1.0f || d < distances[x-1])) {
            distances[x] = distances[x-1];
            labels[x] = labels[x-1];
            x--;
        }
        distances[x] = d;
        labels[x] = local_labels[i];
    }
}

```

```

    }
    free(local_labels);
    free(local_distances);
}

// Classificação final por frequência O(k)
int counts[256] = {0};
for (int i = 0; i < k; i++) if (labels[i] != -1) counts[(unsigned char)labels[i]]++;

char most_frequent = -1;
int max_count = -1;
for (int i = 0; i < 256; i++) {
    if (counts[i] > max_count) {
        max_count = counts[i];
        most_frequent = (char)i;
    }
}

free(labels); free(distances);
return most_frequent;
}
...

---
```

2. Respostas às Questões do Trabalho

(1) Quais dados devem ser enviados pelo mestre?

No modelo OpenMP (memória compartilhada), o "envio" de dados ocorre via definição de escopo. O mestre deve disponibilizar para todas as threads trabalhadoras:

O ponto a ser classificado (`to_evaluate``).

O ponteiro para a estrutura de dados contendo todos os grupos e seus respectivos pontos (`groups``).

O número de vizinhos desejado (`k`) e o total de grupos (`n\_groups`).

Essas variáveis são tratadas como `shared` para que as threads acessem a mesma região de memória sem cópias desnecessárias.

(2) Quais dados devem ser retornados pelos trabalhadores?

Para minimizar a comunicação e evitar regiões críticas constantes, cada trabalhador processa uma fatia dos grupos e retorna um Ranking Top-K Local. Este ranking consiste em dois vetores: um com as `k` menores distâncias encontradas por aquela thread e outro com as `labels` correspondentes.

(3) Como o qsort pode ser paralelizado? Usando apenas OpenMP é suficiente?

O `qsort` pode ser paralelizado através da estratégia de "divisão e conquista" usando OpenMP Tasks. O algoritmo divide o vetor em dois em torno de um pivô; cada uma dessas partições pode ser processada por uma tarefa paralela (`pragma omp task`). Sim, o OpenMP é suficiente, pois suporta recursividade paralela. Entretanto, para este KNN específico, a inserção ordenada durante o processamento ou o uso de um vetor de frequências ao final mostrou-se mais eficiente que ordenar o vetor inteiro.

(4) Como avaliar o desempenho?

A avaliação deve ser feita comparando o tempo de execução da versão sequencial (T_s) contra a versão paralela (T_p) com diferentes quantidades de threads. As métricas são:

Tempo de Execução: Medido com `omp\_get\_wtime()`, focando apenas no core do algoritmo (excluindo leitura de arquivos).

Speedup: $S = T_s / T_p$.

Eficiência: $E = S / \text{number_of_threads}$.

O objetivo é observar um Speedup próximo ao linear conforme aumentamos as threads em datasets grandes.

(5) Sugestão de dataset

Para testar o limite do desempenho, sugere-se a geração de um arquivo com 10.000 grupos e um valor de K alto:

```
```bash
python3 main.py 10000 100 400 60000
```
```

Esta configuração garante carga de trabalho suficiente para que o overhead de criação das threads seja desprezível frente ao processamento.

3. Conclusão

A paralelização com OpenMP utilizando rankings locais e redução via região crítica ao final do loop provou ser uma estratégia eficaz para o KNN. O uso de ``schedule(dynamic)`` compensa possíveis desequilíbrios na quantidade de pontos entre diferentes grupos, garantindo que todos os núcleos do processador sejam utilizados de forma otimizada.

Relatório gerado como parte do trabalho de Programação Paralela.